Date :

Unit – I
Practice – 1

AIM:
To write a C program to implement collision resolution techniques of Hash data structure using Separate Chaining, Linear Probing, and Quadratic Probing.

DESCRIPTION:
A Hash Table is a data structure used to store data efficiently by mapping keys to indices using a hash function. A common hash function is:

h(key) = key % table_size

Sometimes multiple keys map to the same index, causing a collision. To handle this, collision resolution techniques are used.
In Separate Chaining, each index stores a linked list of elements.
In Linear Probing, the next available slot is searched sequentially.
In Quadratic Probing, positions are checked using a quadratic function.
These methods ensure efficient insertion, searching, and deletion operations.

PROCEDURE:
1. Start the program.
2. Define the hash table size.
3. Initialize all entries as empty.
4. Read the key from the user.
5. Compute index using hash function.
6. If collision occurs, apply:
   - Linear Probing
   - Quadratic Probing
   - Separate Chaining
7. Perform insert, search, delete, and display operations.
8. Repeat until exit is selected.
9. Stop the program.

Time Complexity

Insertion : **O(1)**
Search : **O(1)**
Deletion : **O(1)**
Space Complexity: O(n)

PROGRAM:
```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
#define TABLE_SIZE 11
```

Date :

```
#define EMPTY_SLOT INT_MIN
#define DELETED_SLOT (INT_MIN+1)
typedef struct Node{
int key;
struct Node*next;
}Node;
Node*chain_table[TABLE_SIZE];

void init_chaining(){
for(int i=0;i<TABLE_SIZE;i++)chain_table[i]=NULL;
}

int hash_fn(int key){
int h=key%TABLE_SIZE;
if(h<0)h+=TABLE_SIZE;
return h;
}

void chaining_insert(int key){
int idx=hash_fn(key);
Node*newnode=malloc(sizeof(Node));
if(!newnode){printf("Memory error\n");return;}
newnode->key=key;
newnode->next=chain_table[idx];
chain_table[idx]=newnode;
printf("Inserted %d at chain index %d\n",key,idx);
}

Node*chaining_search_node(int key,int*out_idx){
int idx=hash_fn(key);
Node*cur=chain_table[idx];
while(cur){
if(cur->key==key){
if(out_idx)*out_idx=idx;
return cur;
}
cur=cur->next;
}
return NULL;
}

void chaining_delete(int key){
```

Date :

```
int idx=hash_fn(key);
Node*cur=chain_table[idx],*prev=NULL;
while(cur){
if(cur->key==key){
if(prev)prev->next=cur->next;
else chain_table[idx]=cur->next;
free(cur);
printf("Deleted %d from chain index %d\n",key,idx);
return;
}
prev=cur;
cur=cur->next;
}
printf("%d not found in chaining table.\n",key);
}

void chaining_display(){
for(int i=0;i<TABLE_SIZE;i++){
printf("[%2d]:",i);
Node*cur=chain_table[i];
while(cur){
printf(" -> %d",cur->key);
cur=cur->next;
}
printf(" -> NULL\n");
}
}

int oa_table[TABLE_SIZE];

void init_open_addressing(){
for(int i=0;i<TABLE_SIZE;i++)oa_table[i]=EMPTY_SLOT;
}

int linear_insert(int key){
int idx=hash_fn(key);
for(int i=0;i<TABLE_SIZE;i++){
int pos=(idx+i)%TABLE_SIZE;
if(oa_table[pos]==EMPTY_SLOT||oa_table[pos]==DELETED_SLOT){
oa_table[pos]=key;
printf("Inserted %d at index %d (linear)\n",key,pos);
return 1;
```

Date :

```
}
}
printf("Table full cannot insert %d (linear)\n",key);
return 0;
}
int linear_search(int key){
int idx=hash_fn(key);
for(int i=0;i<TABLE_SIZE;i++){
int pos=(idx+i)%TABLE_SIZE;
if(oa_table[pos]==EMPTY_SLOT)return -1;
if(oa_table[pos]==key)return pos;
}
return -1;
}

int linear_delete(int key){
int pos=linear_search(key);
if(pos==-1){
printf("%d not found (linear)\n",key);
return 0;
}
oa_table[pos]=DELETED_SLOT;
printf("Deleted %d from index %d (linear)\n",key,pos);
return 1;
}

int quadratic_insert(int key){
int idx=hash_fn(key);
for(int i=0;i<TABLE_SIZE;i++){
int pos=(idx+i*i)%TABLE_SIZE;
if(oa_table[pos]==EMPTY_SLOT||oa_table[pos]==DELETED_SLOT){
oa_table[pos]=key;
printf("Inserted %d at index %d (quadratic i=%d)\n",key,pos,i);
return 1;
}
}
printf("Table full cannot insert %d (quadratic)\n",key);
return 0;
}

int quadratic_search(int key){
int idx=hash_fn(key);
```

Date :

```
for(int i=0;i<TABLE_SIZE;i++){
int pos=(idx+i*i)%TABLE_SIZE;
if(oa_table[pos]==EMPTY_SLOT)return -1;
if(oa_table[pos]==key)return pos;
}
return -1;
}
int quadratic_delete(int key){
int pos=quadratic_search(key);
if(pos==-1){
printf("%d not found (quadratic)\n",key);
return 0;
}
oa_table[pos]=DELETED_SLOT;
printf("Deleted %d from index %d (quadratic)\n",key,pos);
return 1;
}

void open_addressing_display(){
printf("Index : Value\n");
for(int i=0;i<TABLE_SIZE;i++){
if(oa_table[i]==EMPTY_SLOT)printf("[%2d] : EMPTY\n",i);
else if(oa_table[i]==DELETED_SLOT)printf("[%2d] : DELETED\n",i);
else printf("[%2d] : %d\n",i,oa_table[i]);
}
}

void flush_stdin(){
int c;
while((c=getchar())!='\n'&&c!=EOF){}
}

void show_main_menu(){
printf("\nChoose collision resolution technique:\n");
printf("1. Separate chaining\n");
printf("2. Linear probing (open addressing)\n");
printf("3. Quadratic probing (open addressing)\n");
printf("0. Exit\n");
printf("Selection: ");
void chaining_menu(){
int ch,key;
while(1){
```

Date :

```
printf("\n-- Chaining Menu --\n1: Insert 2: Search 3: Delete 4: Display 0: Back\nChoice: ");
if(scanf("%d",&ch)!=1){flush_stdin();continue;}
if(ch==0)break;
if(ch==1){printf("Key to insert: ");scanf("%d",&key);chaining_insert(key);}
else if(ch==2){printf("Key to search: ");scanf("%d",&key);int
idx;Node*n=chaining_search_node(key,&idx);if(n)printf("Found %d in chain
%d\n",key,idx);else printf("%d not found\n",key);}
else if(ch==3){printf("Key to delete: ");scanf("%d",&key);chaining_delete(key);}
else if(ch==4)chaining_display();
else printf("Invalid choice\n");
}
}
void open_addressing_menu(int mode){
int ch,key;
while(1){
printf("\n-- %s Menu --\n1: Insert 2: Search 3: Delete 4: Display 0: Back\nChoice:
",mode==1?"Linear probing":"Quadratic probing");
if(scanf("%d",&ch)!=1){flush_stdin();continue;}
if(ch==0)break;
if(ch==1){printf("Key to insert: ");scanf("%d",&key);if(mode==1)linear_insert(key);else
quadratic_insert(key);}
else if(ch==2){printf("Key to search: ");scanf("%d",&key);int
pos=(mode==1)?linear_search(key):quadratic_search(key);if(pos==-1)printf("%d not
found\n",key);else printf("Found %d at index %d\n",key,pos);}
else if(ch==3){printf("Key to delete: ");scanf("%d",&key);if(mode==1)linear_delete(key);else
quadratic_delete(key);}
else if(ch==4)open_addressing_display();
else printf("Invalid choice\n");
}
}

int main(){
int sel;
while(1){
show_main_menu();
if(scanf("%d",&sel)!=1){flush_stdin();continue;}
if(sel==0){printf("Goodbye!\n");break;}
if(sel==1){
init_chaining();
printf("\nInitialized separate chaining table (size %d)\n",TABLE_SIZE);
chaining_menu();
for(int i=0;i<TABLE_SIZE;i++){
```

Date :

```
Node*cur=chain_table[i];
while(cur){Node*tmp=cur;cur=cur->next;free(tmp);}
chain_table[i]=NULL;
}
}
else if(sel==2){
init_open_addressing();
printf("\nInitialized open addressing table (size %d) for linear probing\n",TABLE_SIZE);
open_addressing_menu(1);
}
else if(sel==3){
init_open_addressing();
printf("\nInitialized open addressing table (size %d) for quadratic probing\n",TABLE_SIZE);
open_addressing_menu(2);
}
else printf("Invalid selection. Try again.\n");
}
return 0;
}
```

OUTPUT:

```
Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
1

1.Insert
2.Search
3.Delete
4.Display

Enter key: 25
Inserted 25 in chain index 5

Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
1

1.Insert
2.Search
3.Delete
4.Display
4

0 : NULL
1 : NULL
2 : NULL
3 : NULL
4 : NULL
5 : 25 -> NULL
6 : NULL
7 : NULL
8 : NULL
9 : NULL
```

```
Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
2

1.Insert
2.Search
3.Delete
4.Display

Enter key: 45
Inserted at 5

Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
2

1.Insert
2.Search
3.Delete
4.Display
4

[0] EMPTY
[1] EMPTY
[2] EMPTY
[3] EMPTY
[4] EMPTY
[5] 45
[6] EMPTY
[7] EMPTY
[8] EMPTY
[9] EMPTY
```

RESULT:

Thus, the program to implement hashing using Separate Chaining, Linear Probing, and Quadratic Probing with insertion, searching, deletion, and display operations was successfully implemented and executed.

Date :